

Technische Analyse - Kleding Webshop

1. Scope

In Scope

- Zoeken en filteren van producten op basis van diverse criteria (bv. categorie, maat, kleur, prijs).
- Productdetailpagina met uitgebreide informatie (afbeeldingen, beschrijving, specificaties, prijs).
- Functionaliteit voor het toevoegen en beheren van producten in de winkelmand.
- Gestroomlijnd checkoutproces, inclusief adres- en verzendinginformatie.
- Integratie met betaalproviders voor veilige transactieverwerking.
- Gebruikersaccountbeheer (registratie, login, profielbeheer).
- Orderhistoriek voor klanten om eerdere bestellingen te raadplegen.
- Admin productbeheer voor het toevoegen, bewerken en verwijderen van producten.
- Voorraadbeheer om de beschikbaarheid van producten bij te houden.
- Functionaliteit voor het indienen van retouraanvragen door klanten.

Out of Scope

- Integratie met fysieke winkelkassasystemen.
- Implementatie van een loyalty programma of spaarpunten systeem.
- Ondersteuning voor een marketplace model met externe verkopers.
- Ontwikkeling van een geavanceerde AI-gestuurde styling assistant.

2. Assumptions

- De definitie van 'actieve' kledingproducten is gebaseerd op een statusveld in de productdatabase (bv. 'active', 'inactive', 'discontinued').
- Naast Bancontact en Card worden de volgende betaalmethoden ondersteund: iDEAL, PayPal, en Creditcard (Visa, Mastercard, American Express).
- De maximale termijn voor het indienen van een retouraanvraag na levering is 30 dagen, tenzij anders gespecificeerd in de algemene voorwaarden.
- Annuleringen van bestellingen door de klant zijn mogelijk tot het moment van verzending. Annuleringen door de admin zijn altijd mogelijk, met automatische terugbetaling.
- JWT tokens zullen worden beveiligd met een sterke, geheime sleutel en een redelijke vervaltijd (bv. 1 uur voor access tokens, 24 uur voor refresh tokens).
- De voorraad wordt beheerd via een 'first-come, first-served' mechanisme. Bij gelijktijdige bestellingen wordt de voorraad gereserveerd op het moment van checkout voltooiing, en wordt de eerstvolgende bestelling geweigerd indien de voorraad ontoereikend is.

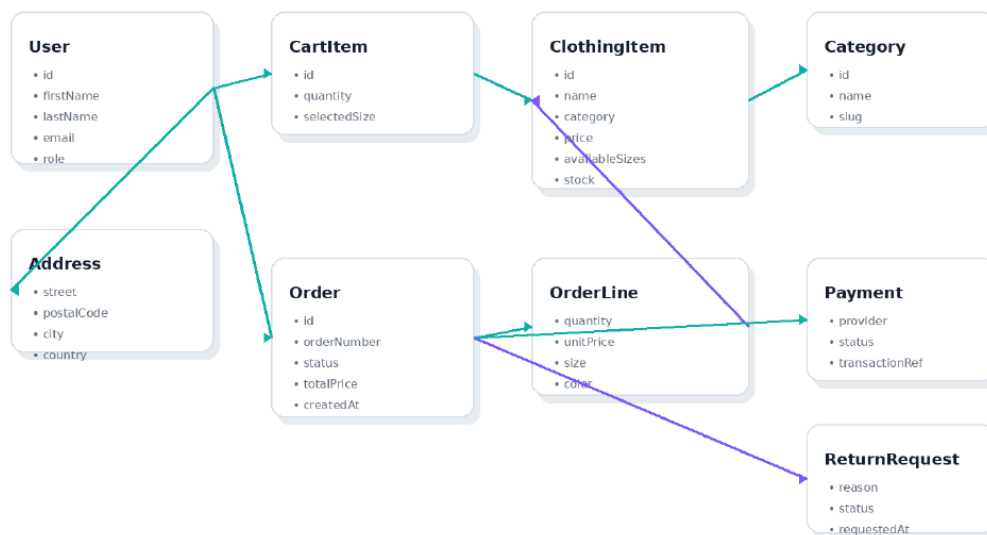
3. Open Questions

- Wat is de exacte definitie van 'actieve' kledingproducten? (bv. welke statusvelden worden gebruikt, zijn er uitzonderingen?)
- Welke specifieke betaalmethoden worden ondersteund naast Bancontact en Card? (bv. iDEAL, PayPal, specifieke creditcardmaatschappijen?)
- Wat is de maximale termijn voor het indienen van een retouraanvraag na levering (naast de 14 dagen genoemd in BR-007)?
- Hoe wordt omgegaan met het annuleren van bestellingen door de klant of admin? (bv. welke workflows, notificaties, terugbetalingsmechanismen?)
- Zijn er specifieke eisen voor de beveiliging van de JWT tokens? (bv. encryptie, opslag, rotatiebeleid?)
- Hoe wordt de voorraad beheerd bij meerdere gelijktijdige bestellingen van hetzelfde item? (bv. locking mechanismen, race conditions, notificaties bij uitverkocht?)

4. Domain Model

6. Domain model

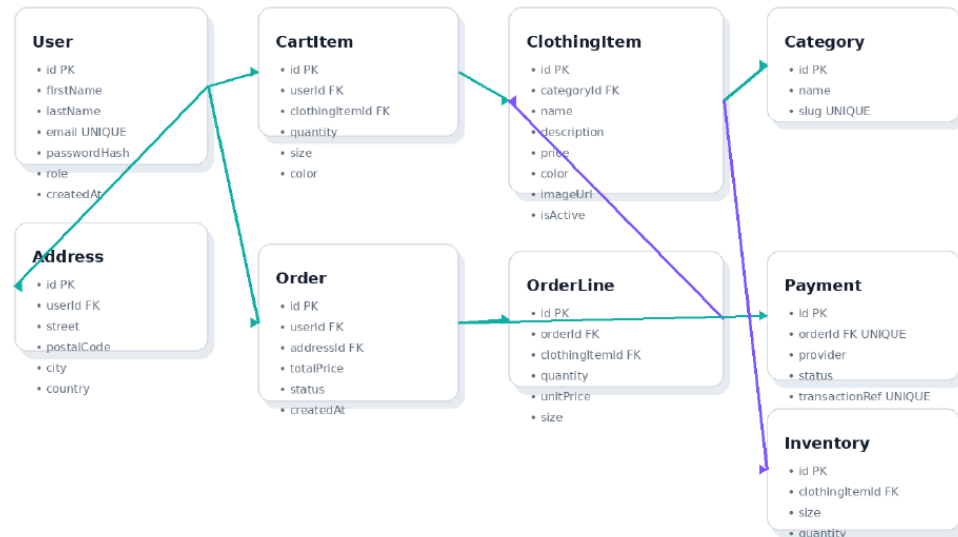
Het domeinmodel beschrijft de belangrijkste concepten binnen de kledingwebshop en hun relaties. Het model focust op betekenisvolle businessobjecten, niet op technische database details.



Domeinconcept	Beschrijving
User	Een geregistreerde gebruiker met klant- of adminrol.
ClothingItem	Een verkoopbaar kledingstuk met naam, prijs, categorie, kleur en beschikbare maten.
Category	Groepering zoals Hoodies, Jeans, Jackets of Accessories.
CartItem	Een gekozen product in het winkelmandje met maat, kleur en hoeveelheid.
Order	Een geplaatste bestelling met status, totaalprijs en verzendadres.
OrderLine	Een individuele lijn binnen een bestelling.
Payment	De betaling die gekoppeld is aan exact een bestelling.
ReturnRequest	Een aanvraag om een geleverd product te retourneren.

7. Database ERD

De database bestaat uit gebruikers, adressen, categorieën, kledingstukken, voorraad, winkelmanditems, bestellingen, orderregels en betalingen. Belangrijke relaties zijn User 1-N Order, Order 1-N OrderLine, ClothingItem 1-N OrderLine, User 1-N CartItem en Order 1-1 Payment.



Entiteit	Belangrijkste velden	Constraints
User	id, firstName, lastName, email, passwordHash, role, createdAt	email uniek; passwordHash verplicht; role in Customer/Admin
ClothingItem	id, categoryId, name, description, price, color, imageUrl, isActive	price > 0; categoryId verplicht
Inventory	id, clothingItemId, size, quantity	quantity >= 0; unieke combinatie clothingItemId + size
CartItem	id, userId, clothingItemId, quantity, size, color	quantity > 0; quantity mag voorraad niet overschrijden
Order	id, userId, addressId, totalPrice, status, createdAt	status in Pending/Paid/Shipped/Delivered/Cancelled
Payment	id, orderId, provider, status, transactionRef	orderId uniek; transactionRef uniek
ReturnRequest	id, orderId, reason, status, requestedAt	alleen mogelijk voor geleverde bestellingen

User

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
username	String	notNull, minLength:3, maxLength:50	empty, too_short, too_long, missing, invalid_value, duplicate_per_day
password	String	notNull, minLength:8	empty, too_short, missing, invalid_value
email	String	notNull, maxLength:255	empty, too_long, missing, invalid_value, duplicate_per_day
role	UserRole	notNull	missing, invalid_value
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value

UserRole

Veld	Type	Constraints	Testcases
value	enum_value	notNull	missing, invalid_value

ClothingItem

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
name	String	notNull, maxLength:255	empty, too_long, missing, invalid_value
description	String		empty, invalid_value
price	BigDecimal	notNull	missing, invalid_value
category	Category	notNull	missing, invalid_value
availableSizes	List	notNull	missing, invalid_value
availableColors	List	notNull	missing, invalid_value
inventory	Inventory	notNull	missing, invalid_value
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value

Category

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
name	String	notNull, maxLength:100	empty, too_long, missing, invalid_value, duplicate_per_day
description	String		empty, invalid_value
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value

CartItem

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
user	User	notNull	missing, invalid_value
clothingItem	ClothingItem	notNull	missing, invalid_value
size	String	notNull	empty, missing, invalid_value

Veld	Type	Constraints	Testcases
color	String	notNull	empty, missing, invalid_value
quantity	Integer	notNull, min:1	missing, invalid_value, too_short
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value

Order

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
user	User	notNull	missing, invalid_value
orderLines	List	notNull	missing, invalid_value
status	OrderStatus	notNull	missing, invalid_value
totalPrice	BigDecimal	notNull	missing, invalid_value
shippingAddress	Address	notNull	missing, invalid_value
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value
deliveredAt	LocalDateTime		invalid_value

OrderStatus

Veld	Type	Constraints	Testcases
value	enum_value	notNull	missing, invalid_value

OrderLine

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
order	Order	notNull	missing, invalid_value
clothingItem	ClothingItem	notNull	missing, invalid_value
size	String	notNull	empty, missing, invalid_value
color	String	notNull	empty, missing, invalid_value
quantity	Integer	notNull, min:1	missing, invalid_value, too_short
pricePerUnit	BigDecimal	notNull	missing, invalid_value
createdAt	LocalDateTime	notNull	missing, invalid_value

Veld	Type	Constraints	Testcases
updatedAt	LocalDateTime	notNull	missing, invalid_value

Payment

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
order	Order	notNull	missing, invalid_value
paymentMethod	String	notNull, maxLength:50	empty, too_long, missing, invalid_value
transactionId	String	notNull, maxLength:255	empty, too_long, missing, invalid_value
amount	BigDecimal	notNull	missing, invalid_value
status	PaymentStatus	notNull	missing, invalid_value
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value

PaymentStatus

Veld	Type	Constraints	Testcases
value	enum_value	notNull	missing, invalid_value

ReturnRequest

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
order	Order	notNull	missing, invalid_value
reason	String	notNull, maxLength:1000	empty, too_long, missing, invalid_value
status	ReturnRequestStatus	notNull	missing, invalid_value
requestedAt	LocalDateTime	notNull	missing, invalid_value
processedAt	LocalDateTime		invalid_value
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value

ReturnRequestStatus

Veld	Type	Constraints	Testcases
value	enum_value	notNull	missing, invalid_value

Address

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
street	String	notNull, maxLength:255	empty, too_long, missing, invalid_value
city	String	notNull, maxLength:100	empty, too_long, missing, invalid_value
postalCode	String	notNull, maxLength:20	empty, too_long, missing, invalid_value
country	String	notNull, maxLength:100	empty, too_long, missing, invalid_value
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value

Inventory

Veld	Type	Constraints	Testcases
id	UUID	notNull	missing, invalid_value
clothingItem	ClothingItem	notNull	missing, invalid_value
size	String	notNull	empty, missing, invalid_value
color	String	notNull	empty, missing, invalid_value
quantity	Integer	notNull, min:0	missing, invalid_value, too_short
createdAt	LocalDateTime	notNull	missing, invalid_value
updatedAt	LocalDateTime	notNull	missing, invalid_value

Enums

- **UserRole:** (e.g., ADMIN, CUSTOMER)
- **OrderStatus:** (e.g., PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED)
- **PaymentStatus:** (e.g., PENDING, SUCCESS, FAILED, REFUNDED)
- **ReturnRequestStatus:** (e.g., PENDING, APPROVED, REJECTED, COMPLETED)

5. API Design

5.1 Error Formaat

```
{
  "correlationId": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
  "code": "INVALID_INPUT",
  "message": "De opgegeven input is ongeldig.",
  "fieldErrors": [
    {
      "field": "email",
      "message": "Ongeldig e-mailformaat."
    }
  ]
}
```

5.2 Endpoints

GET /api/clothes

Haal een lijst met kledingitems op, met filter- en sorteeropties.

| Veld | Waarde

6. Backend Design

De backend van de kleding webshop volgt een gelaagde architectuur, bestaande uit de volgende lagen:

- **Controller Laag:** Verantwoordelijk voor het afhandelen van inkomende HTTP-verzoeken, het valideren van de invoer en het doorsturen van verzoeken naar de service laag.
- **Service Laag:** Bevat de kern bedrijfslogica. Deze laag orkestreert operaties, communiceert met de repository laag en implementeert de business rules.
- **Repository Laag:** Verantwoordelijk voor de interactie met de datastore (database). Deze laag voert CRUD-operaties uit op de domein-entiteiten.

Daarnaast worden Data Transfer Objects (DTO's) gebruikt om gegevens tussen de lagen en de client uit te wisselen, en worden er specifieke exceptions gedefinieerd om foutcondities te signaleren.

Clothing Item Module

Klasse	Verantwoordelijkheid
ClothingItemController	Behandelt HTTP-verzoeken voor kledingitems, inclusief het ophalen van lijsten en details.

Klasse	Verantwoordelijkheid
<code>ClothingItemService</code>	Beheert de bedrijfslogica voor kledingitems, inclusief filtering, sortering en voorraadbeheer.
<code>ClothingItemRepository</code>	Verantwoordelijk voor de interactie met de database voor kledingitem-entiteiten.
<code>ClothingItem</code>	Representeert een kledingitem in het domeinmodel.
<code>ClothingItemDTO</code>	Data Transfer Object voor kledingitems, gebruikt voor API-communicatie.
<code>PaginatedClothingItemListResponseDTO</code>	DTO voor de gepagineerde lijst van kledingitems.
<code>ClothingItemDetailResponseDTO</code>	DTO voor de details van een enkel kledingitem.
<code>CreateClothingItemRequestDTO</code>	DTO voor het aanmaken van een nieuw kledingitem.
<code>UpdateClothingItemRequestDTO</code>	DTO voor het updaten van een bestaand kledingitem.
<code>ClothingItemNotFoundException</code>	Exception die wordt gegooid wanneer een kledingitem niet wordt gevonden.
<code>InsufficientStockException</code>	Exception die wordt gegooid wanneer er onvoldoende voorraad is voor een kledingitem.
<code>ClothingItemValidator</code>	Valideert de invoer voor kledingitem-gerelateerde operaties.

Category Module

Klasse	Verantwoordelijkheid
<code>CategoryService</code>	Beheert de bedrijfslogica voor categorieën.
<code>CategoryRepository</code>	Verantwoordelijk voor de interactie met de database voor categorie-entiteiten.
<code>Category</code>	Representeert een categorie in het domeinmodel.
<code>CategoryDTO</code>	Data Transfer Object voor categorieën.

Inventory Module

Klasse	Verantwoordelijkheid
<code>InventoryService</code>	Beheert de bedrijfslogica voor voorraad.
<code>InventoryRepository</code>	Verantwoordelijk voor de interactie met de database voor voorraad-entiteiten.
<code>Inventory</code>	Representeert de voorraad van een kledingitem voor een specifieke maat en kleur.
<code>InventoryDTO</code>	Data Transfer Object voor voorraad.

Auth Module

Klasse	Verantwoordelijkheid
<code>AuthController</code>	Behandelt HTTP-verzoeken voor gebruikersregistratie en -login.
<code>AuthService</code>	Beheert de bedrijfslogica voor authenticatie, inclusief gebruikersregistratie, login en JWT-generatie.
<code>UserRepository</code>	Verantwoordelijk voor de interactie met de database voor gebruikersentiteiten.
<code>User</code>	Representeert een gebruiker in het domeinmodel.
<code>UserRole</code>	Enum voor de rollen van gebruikers (bv. CUSTOMER, ADMIN).
<code>RegisterUserRequestDTO</code>	DTO voor het registreren van een nieuwe gebruiker.
<code>LoginUserRequestDTO</code>	DTO voor het inloggen van een gebruiker.
<code>AuthResponseDTO</code>	DTO voor het antwoord na succesvolle authenticatie, inclusief JWT.
<code>EmailAlreadyExistsException</code>	Exception die wordt gegooid wanneer een e-mailadres al in gebruik is.
<code>InvalidCredentialsException</code>	Exception die wordt gegooid bij ongeldige inloggegevens.
<code>UserValidator</code>	Valideert de invoer voor gebruikersgerelateerde operaties.
<code>PasswordHasher</code>	Verantwoordelijk voor het hashen en verifiëren van wachtwoorden.

Cart Module

Klasse	Verantwoordelijkheid
<code>CartController</code>	Behandelt HTTP-verzoeken voor het winkelmandje.
<code>CartService</code>	Beheert de bedrijfslogica voor het winkelmandje, inclusief toevoegen, bijwerken en verwijderen van items.
<code>CartItemRepository</code>	Verantwoordelijk voor de interactie met de database voor winkelmandje-item-entiteiten.
<code>CartItem</code>	Representeert een item in het winkelmandje.
<code>CartResponseDTO</code>	DTO voor het antwoord van het winkelmandje.
<code>CartItemResponseDTO</code>	DTO voor een enkel item in het winkelmandje.
<code>AddCartItemRequestDTO</code>	DTO voor het toevoegen van een item aan het winkelmandje.
<code>UpdateCartItemRequestDTO</code>	DTO voor het bijwerken van een item in het winkelmandje.
<code>CartItemNotFoundException</code>	Exception die wordt gegooid wanneer een winkelmandje-item niet wordt gevonden.
<code>CartValidator</code>	Valideert de invoer voor winkelmandje-gerelateerde operaties.

Order Module

Klasse	Verantwoordelijkheid
<code>OrderController</code>	Behandelt HTTP-verzoeken voor bestellingen.
<code>OrderService</code>	Beheert de bedrijfslogica voor bestellingen, inclusief creatie, statusupdates en het ophalen van bestelgeschiedenis.
<code>OrderRepository</code>	Verantwoordelijk voor de interactie met de database voor bestelling-entiteiten.
<code>OrderLineRepository</code>	Verantwoordelijk voor de interactie met de database voor bestellijn-entiteiten.
<code>Order</code>	Representeert een bestelling in het domeinmodel.
<code>OrderLine</code>	Representeert een regel binnen een bestelling.
<code>OrderStatus</code>	Enum voor de statussen van een bestelling (bv. PENDING, PAID, DELIVERED).
<code>OrderResponseDTO</code>	DTO voor het antwoord van een bestelling.
<code>OrderListResponseDTO</code>	DTO voor een lijst met bestellingen.
<code>CreateOrderRequestDTO</code>	DTO voor het aanmaken van een nieuwe bestelling.
<code>UpdateOrderStatusRequestDTO</code>	DTO voor het bijwerken van de status van een bestelling.
<code>OrderNotFoundException</code>	Exception die wordt gegooid wanneer een bestelling niet wordt gevonden.
<code>EmptyCartException</code>	Exception die wordt gegooid wanneer een bestelling wordt aangemaakt met een leeg winkelmandje.
<code>OrderCreationConflictException</code>	Exception die wordt gegooid bij conflicten tijdens het aanmaken van een bestelling (bv. voorraadprobleem).
<code>OrderValidator</code>	Valideert de invoer voor bestelling-gerelateerde operaties.

Payment Module

Klasse	Verantwoordelijkheid
<code>PaymentController</code>	Behandelt HTTP-verzoeken voor betalingen, inclusief webhooks.
<code>PaymentService</code>	Beheert de bedrijfslogica voor betalingen, inclusief het verwerken van webhooks en het bijwerken van orderstatussen.
<code>PaymentRepository</code>	Verantwoordelijk voor de interactie met de database voor betaling-entiteiten.
<code>Payment</code>	Representeert een betaling in het domeinmodel.
<code>PaymentStatus</code>	Enum voor de statussen van een betaling (bv. SUCCESS, FAILED).

Klasse	Verantwoordelijkheid
<code>PaymentWebhookRequestDTO</code>	DTO voor de payload van een betalingswebhook.
<code>PaymentProcessingException</code>	Exception die wordt gegooid tijdens het verwerken van een betaling.

Returns Module

Klasse	Verantwoordelijkheid
<code>ReturnController</code>	Behandelt HTTP-verzoeken voor retouraanvragen.
<code>ReturnService</code>	Beheert de bedrijfslogica voor retouraanvragen, inclusief het indienen en verwerken ervan.
<code>ReturnRequestRepository</code>	Verantwoordelijk voor de interactie met de database voor retouraanvraag-entiteiten.
<code>ReturnRequest</code>	Representeert een retouraanvraag in het domeinmodel.
<code>ReturnRequestStatus</code>	Enum voor de statussen van een retouraanvraag (bv. PENDING, APPROVED, REJECTED).
<code>CreateReturnRequestDTO</code>	DTO voor het indienen van een retouraanvraag.
<code>ReturnRequestResponseDTO</code>	DTO voor het antwoord van een retouraanvraag.
<code>ReturnRequestNotAllowedException</code>	Exception die wordt gegooid wanneer een retouraanvraag niet is toegestaan.
<code>ReturnRequestValidator</code>	Valideert de invoer voor retouraanvraag-gerelateerde operaties.

Address Module

Klasse	Verantwoordelijkheid
<code>AddressService</code>	Beheert de bedrijfslogica voor adressen.
<code>AddressRepository</code>	Verantwoordelijk voor de interactie met de database voor adres-entiteiten.
<code>Address</code>	Representeert een adres in het domeinmodel.
<code>AddressDTO</code>	Data Transfer Object voor adressen.

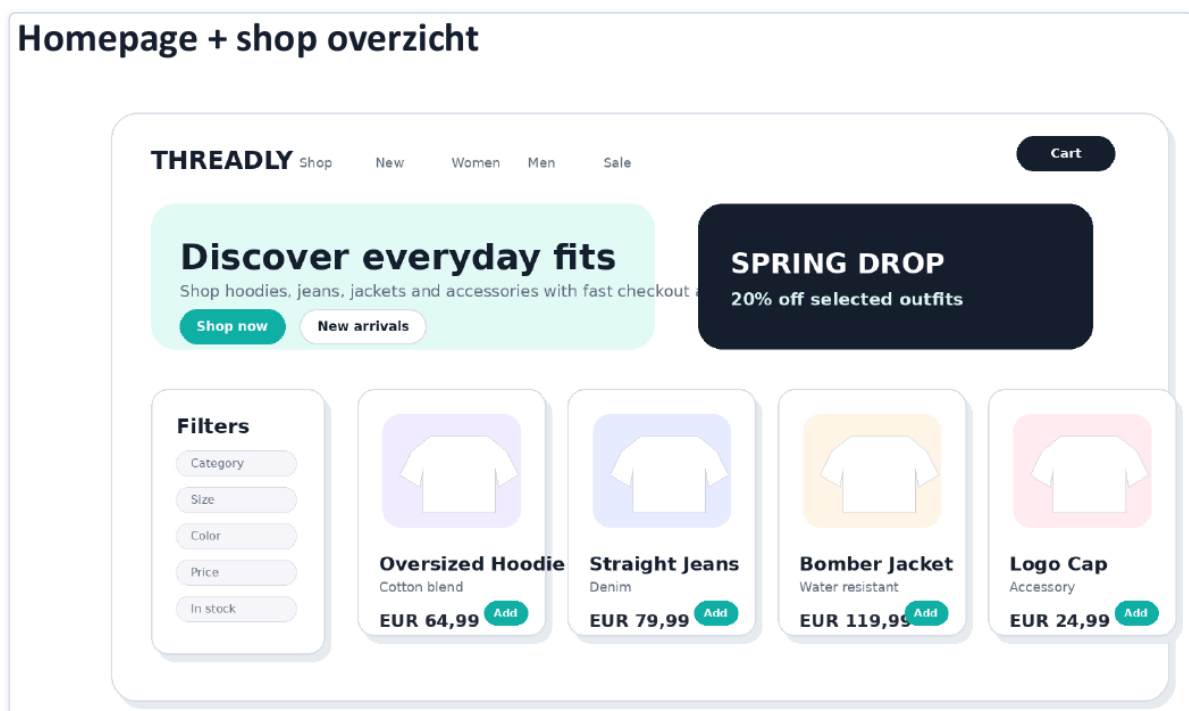
Common Module

Klasse	Verantwoordelijkheid
<code>ApiError</code>	Standaardformaat voor API-foutmeldingen.
<code>ApiErrorDTO</code>	DTO voor de standaard API-foutmelding.
<code>GlobalExceptionHandler</code>	Centrale handler voor het afhandelen van exceptions en het retourneren van API-foutmeldingen.

Klasse	Verantwoordelijkheid
<code>SecurityConfig</code>	Configureert beveiligingsinstellingen, inclusief authenticatie en autorisatie.
<code>JwtTokenProvider</code>	Verantwoordelijk voor het genereren en valideren van JWT-tokens.
<code>PageableRequest</code>	Standaard object voor paginatie- en sorteerverzoeken.
<code>PageableResponse</code>	Standaard object voor gepagineerde antwoorden.
<code>CorrelationIdFilter</code>	Filter om een correlatie-ID toe te voegen aan verzoeken voor tracing.
<code>ValidationConfig</code>	Configureert validatieregels en beanvalidatie.

7. Frontend Design

Homepage + shop overzicht



Productdetailpagina

THREADLYAccount Cart



Oversized Hoodie - Sand

Heavyweight cotton hoodie with relaxed fit, dropped shoulders and soft

Category **Hoodies**

Size **XS - XXL**

Color **Sand**

Stock **42 Items**

EUR 64,99

Add to cart

Wishlist

Business rule: add-to-cart is disabled when selected size/color is out of stock.

Checkout en order summary

Checkout

Shipping details

First name

Last name

Street + number

Postal code

City

E-mail

Payment method

Bancontact

Card

Order summary

Oversized Hoodie 1x	EUR 64,99
Straight Jeans 1x	EUR 79,99
Delivery Standard	EUR 4,95

Total

EUR 149,93

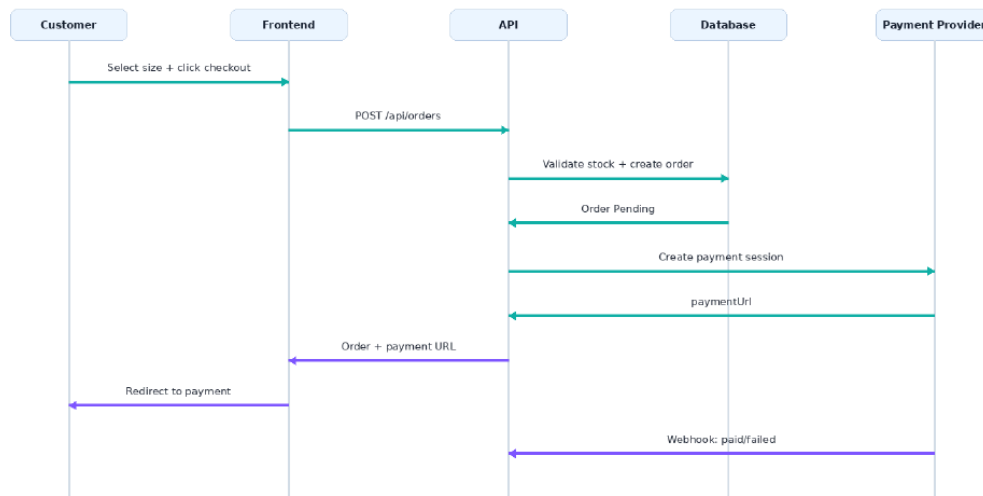
Pay now

5. Uitgebreide UML diagrams

De UML diagrammen ondersteunen de analyse van gedrag, componenten en deployment.

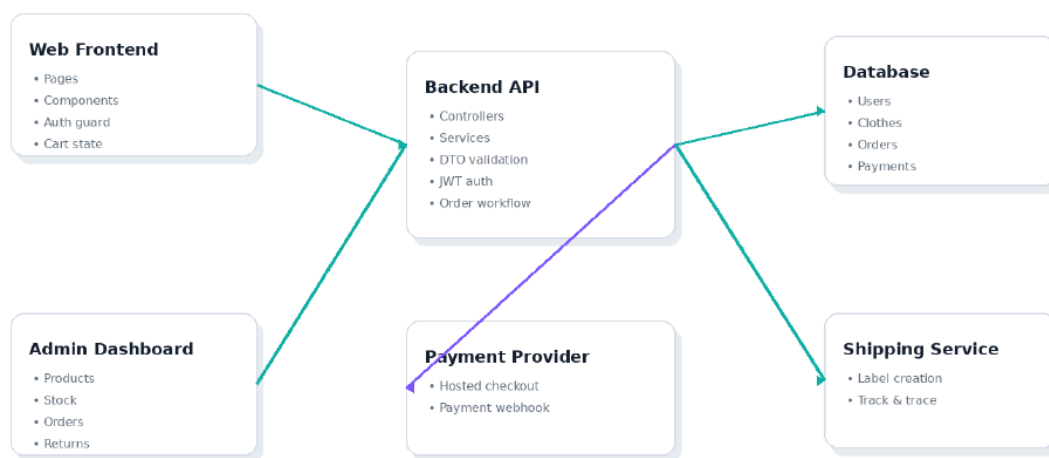
Sequence diagram - checkout en betaling

Deze flow toont hoe klant, frontend, API, database en betalingsprovider samenwerken bij het plaatsen en betalen van een bestelling.



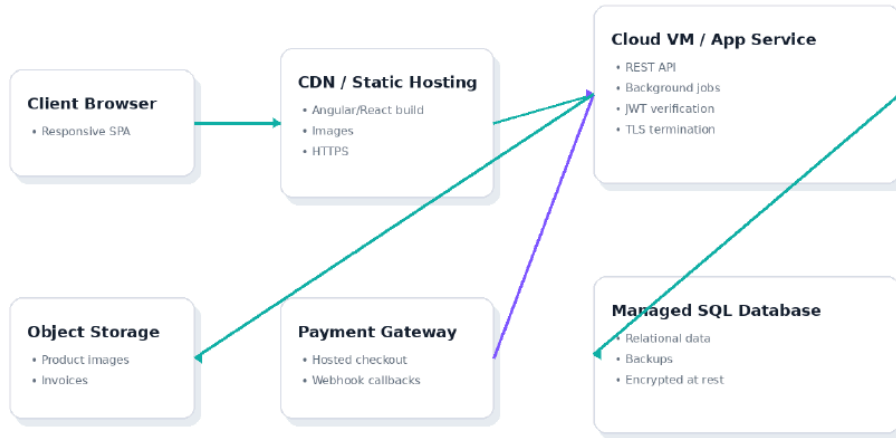
Component diagram

Dit diagram toont de logische applicatiecomponenten en hun verantwoordelijkheden.



Deployment diagram

Dit diagram toont een mogelijke cloud deployment met client, CDN, app service, database, object storage en externe providers.



/

Component	Verantwoordelijkheid
HomePage	Toont de hoofdpagina met een overzicht van kledingproducten en filteropties.
ProductList	Rendert de lijst met kledingitems, inclusief zoek- en filterfunctionaliteit.
ProductFilter	Biedt gebruikers de mogelijkheid om producten te filteren op categorie, maat, kleur, prijs en beschikbaarheid.
Pagination	Beheert de paginatie van de productlijst.

/products/:id

Component	Verantwoordelijkheid
ProductDetailPage	Toont de details van een specifiek kledingitem, inclusief prijs, beschrijving, maten en voorraadstatus.
AddToCartForm	Stelt de gebruiker in staat om een product met gekozen maat en kleur aan het winkelmandje toe te voegen.

/cart

Component	Verantwoordelijkheid
CartPage	Toont de inhoud van het winkelmandje, inclusief items, hoeveelheden en totalen.
CartItem	Rendert een individueel item in het winkelmandje met opties om de hoeveelheid aan te passen of te verwijderen.
CheckoutButton	Navigeert de gebruiker naar de checkoutpagina.

/checkout

Component	Verantwoordelijkheid
CheckoutPage	Begeleidt de gebruiker door het afrekenproces, inclusief verzendgegevens, order summary en betaalmethode.
ShippingForm	Verzamelt de verzendgegevens van de gebruiker.
OrderSummary	Toont een overzicht van de te bestellen items en de totale kosten.
PaymentForm	Verzamelt de betaalgegevens van de gebruiker.
PlaceOrderButton	Dient de bestelling in.

/account/orders

Component	Verantwoordelijkheid
OrderHistoryPage	Toont de bestelgeschiedenis van de ingelogde gebruiker.
OrderList	Rendert de lijst met bestellingen.
OrderItem	Rendert een individuele bestelling met statusinformatie.

/account/orders/:id

Component	Verantwoordelijkheid
OrderDetailPage	Toont de details van een specifieke bestelling.
ReturnRequestForm	Stelt de gebruiker in staat om een retouraanvraag in te dienen voor een bestelling.

/admin/products

Component	Verantwoordelijkheid
AdminProductListPage	Toont een lijst van alle kledingproducten voor beheer.
AdminProductTable	Rendert een tabel met kledingproducten en beheeropties.
AddProductButton	Navigeert naar de pagina voor het toevoegen van een nieuw product.

/admin/products/new

Component	Verantwoordelijkheid
AdminAddProductPage	Formulier voor het toevoegen van een nieuw kledingitem.
AdminProductForm	Formulier voor het invoeren van productgegevens.

/admin/products/:id

Component	Verantwoordelijkheid
AdminEditProductPage	Formulier voor het bewerken van een bestaand kledingitem.
AdminProductForm	Formulier voor het bewerken van productgegevens.

/admin/orders

Component	Verantwoordelijkheid
AdminOrderListPage	Toont een lijst van alle bestellingen voor beheer.
AdminOrderTable	Rendert een tabel met bestellingen en beheeropties.

/admin/orders/:id

Component	Verantwoordelijkheid
AdminOrderDetailPage	Toont de details van een specifieke bestelling voor beheer.
UpdateOrderStatusForm	Formulier voor het bijwerken van de status van een bestelling.

/login

Component	Verantwoordelijkheid
LoginPage	Formulier voor het inloggen van gebruikers.
LoginForm	Verwerkt de inloggegevens.

/register

Component	Verantwoordelijkheid
RegisterPage	Formulier voor het registreren van nieuwe gebruikers.
RegisterForm	Verwerkt de registratiegegevens.

8. Security & Privacy

Authenticatie

- **Klantauthenticatie:** Klanten authenticeren zich via een JWT (JSON Web Token) na succesvolle login via `/api/auth/login`. De token wordt opgeslagen in de `localStorage` van de browser en meegestuurd in de `Authorization: Bearer <token>` header voor beveiligde endpoints.
- **Adminauthenticatie:** Admingebruikers authenticeren zich op dezelfde wijze als klanten. De autorisatie wordt gecontroleerd op basis van de rol die aan de gebruiker is toegekend in het

authenticatiesysteem.

- **Wachtwoordbeveiliging:** Wachtwoorden worden opgeslagen met een sterke hashing-algoritme (bijvoorbeeld bcrypt) met een salt.

Autorisatie

- **Klanttoegang:** Klanten hebben toegang tot de volgende endpoints:
 - `/api/clothes` (GET)
 - `/api/clothes/{id}` (GET)
 - `/api/cart` (GET, POST, PUT, DELETE)
 - `/api/cart/items` (POST, PUT, DELETE)
 - `/api/orders/me` (GET)
 - `/api/returns` (POST)
- **Admin Toegang:** Admingebruikers hebben toegang tot de volgende endpoints:
 - `/api/admin/clothes` (GET, POST, PUT, DELETE)
 - `/api/admin/clothes/{id}` (GET, PUT, DELETE)
 - `/api/admin/orders/{id}/status` (PUT)
 - Alle klant-endpoints (voor beheerdoeleinden, indien nodig).
- **Rolgebaseerde Toegang:** De applicatie implementeert rolgebaseerde toegangscontrole (RBAC). De server valideert de rol van de gebruiker (klant of admin) op basis van de informatie in de JWT voordat toegang wordt verleend tot specifieke endpoints.
- **Orderinzage:** Klanten kunnen alleen hun eigen bestellingen bekijken via `/api/orders/me`.

Privacyoverwegingen

- **Persoonsgegevens:** Persoonsgegevens (zoals naam, adres, e-mail) worden alleen verzameld voor het afhandelen van bestellingen en worden opgeslagen in de database. Toegang tot deze gegevens is strikt beperkt tot geautoriseerd personeel.
- **Betalingsgegevens:** Gevoelige betalingsgegevens worden niet direct in de applicatie opgeslagen. De integratie met de betalingsprovider (bijvoorbeeld Stripe, Mollie) zorgt voor veilige afhandeling van betalingen via tokens.
- **Data Retentie:** Beleid voor data retentie zal worden gedefinieerd voor klantgegevens en orderhistorie.

9. Observability

Logging

Logging zal worden geïmplementeerd op verschillende niveaus (INFO, WARN, ERROR) om de applicatieactiviteit te monitoren en problemen te diagnosticeren.

Concrete Voorbeelden van Logging:

- **Authenticatie:**

- INFO: User 'user@example.com' logged in successfully.
- WARN: Failed login attempt for user 'unknown@example.com' from IP 192.168.1.100.
- ERROR: JWT validation failed for token: <truncated_token_string>.

- **Productbeheer (Admin):**

- INFO: Admin user 'admin@example.com' created new product 'T-Shirt - Blue - M'. Product ID: 12345.
- INFO: Admin user 'admin@example.com' updated product 'Jeans - Black - L'. Product ID: 67890.
- ERROR: Failed to update product 'Sneakers - White - 42' due to database constraint violation. Product ID: 11223.

- **Winkelmandje:**

- INFO: User 'user@example.com' added product 'Hoodie - Grey - L' (Product ID: 54321) to cart.
- INFO: User 'user@example.com' removed product 'Socks - White - One Size' (Product ID: 98765) from cart.
- WARN: Attempted to add out-of-stock item 'Dress - Red - S' (Product ID: 33445) to cart.

- **Bestelproces:**

- INFO: Order created for user 'user@example.com'. Order ID: ORD-1001. Status: Pending.
- INFO: Payment webhook received for Order ID: ORD-1001. Status updated to Paid.
- ERROR: Payment webhook failed for Order ID: ORD-1002. Reason: Invalid signature.
- INFO: Inventory reserved for Order ID: ORD-1003. Product: 'Jacket - Green - XL', Quantity: 1.
- INFO: Inventory reduced for Order ID: ORD-1003 after successful payment. Product: 'Jacket - Green - XL', Quantity: 1.

- **Retouraanvragen:**

- INFO: Return request submitted for Order ID: ORD-1004 by user 'user@example.com'. Reason: 'Item too small'.
- WARN: Return request for Order ID: ORD-1005 rejected. Order not delivered or outside 14-day window.

Metrics

- **Request Latency:** Gemiddelde en percentielen (95th, 99th) van de responstijd voor belangrijke endpoints (bv. `/api/clothes`, `/api/cart`, `/api/orders`).
- **Error Rate:** Aantal 4xx en 5xx HTTP-responses per endpoint.

- **Database Query Performance:** Gemiddelde en percentielen van de uitvoeringstijd van kritieke databasequeries.
- **Inventory Levels:** Aantal beschikbare items per product/variant.
- **Order Throughput:** Aantal succesvol geplaatste bestellingen per tijdseenheid.
- **User Activity:** Aantal actieve gebruikers, aantal logins.

Correlation ID

Een unieke `correlationId` zal worden gegenereerd aan het begin van elke request en meegestuurd in alle logs en interne service-aanroepen. Dit maakt het mogelijk om de volledige levenscyclus van een request te traceren, zelfs over meerdere services heen.

- **Voorbeeld Log met Correlation ID:**

```
[2023-10-27T10:30:00Z] [INFO] [correlationId: abcdef1234567890] User
'user@example.com' added product 'Hoodie - Grey - L' (Product ID: 54321) to cart.
```

- **Voorbeeld Interne Service Call met Correlation ID:**

Wanneer de `/api/orders` endpoint wordt aangeroepen, wordt de `correlationId` meegestuurd naar de inventory service en de payment service.

10. Performance & Scalability

Performance-eisen

- **Productoverzicht (`/api/clothes`):** < 500ms responstijd voor 95% van de requests, zelfs met duizenden producten.
- **Productdetailpagina (`/api/clothes/{id}`):** < 200ms responstijd voor 95% van de requests.
- **Toevoegen aan winkelmandje (`/api/cart/items`):** < 300ms responstijd voor 95% van de requests.
- **Checkout proces:** < 1000ms responstijd voor de initiële ordercreatie.
- **Pagina laden (Frontend):** Alle pagina's moeten binnen 3 seconden volledig geladen zijn op een gemiddelde internetverbinding.

Database-indexen

Om de performance van de database te optimaliseren, zullen de volgende indexen worden aangemaakt:

- **products tabel:**
 - `PRIMARY KEY (id)`
 - `INDEX idx_products_category (category)`
 - `INDEX idx_products_price (price)`
 - `INDEX idx_products_is_active (is_active)`
- **product_variants tabel (voor maat, kleur, voorraad):**

- `PRIMARY KEY (id)`
- `INDEX idx_product_variants_product_id (product_id)`
- `INDEX idx_product_variants_size (size)`
- `INDEX idx_product_variants_color (color)`
- `INDEX idx_product_variants_stock_quantity (stock_quantity)`
- `UNIQUE INDEX uidx_product_variant (product_id, size, color)` (om duplicaten te voorkomen en snelle lookup)
- **orders tabel:**
 - `PRIMARY KEY (id)`
 - `INDEX idx_orders_user_id (user_id)`
 - `INDEX idx_orders_status (status)`
 - `INDEX idx_orders_created_at (created_at)`
- **order_items tabel:**
 - `PRIMARY KEY (id)`
 - `INDEX idx_order_items_order_id (order_id)`
 - `INDEX idx_order_items_product_variant_id (product_variant_id)`
- **users tabel:**
 - `PRIMARY KEY (id)`
 - `UNIQUE INDEX uidx_users_email (email)`

Schaalbaarheid

- **Stateless Services:** De backend services zullen stateless worden ontworpen. Dit betekent dat elke request onafhankelijk kan worden verwerkt zonder afhankelijk te zijn van server-specifieke sessiegegevens. Dit maakt horizontale schaalbaarheid (het toevoegen van meer serverinstanties) eenvoudig.
- **Database Schaalbaarheid:**
 - **Replicatie:** Gebruik maken van read replicas voor de database om de leesbelasting te verdelen.
 - **Sharding:** Indien de dataset extreem groot wordt, kan sharding van de database worden overwogen (bijvoorbeeld sharden op `user_id` voor orders).
- **Caching:**
 - **Productdata:** Veelgevraagde productinformatie kan worden gecached (bv. in Redis) om de database te ontlasten.
 - **Gebruikerssessies:** JWTs kunnen worden gebruikt voor sessiebeheer, wat de noodzaak voor server-side sessieopslag vermindert.
- **Asynchrone Verwerking:** Taken die niet direct een directe gebruikersrespons vereisen (bv. het versturen van e-mails na een bestelling, het verwerken van complexe rapportages) kunnen asynchroon worden afgehandeld via een message queue (bv. RabbitMQ, Kafka).

- **Load Balancing:** Een load balancer zal worden ingezet om verkeer te verdelen over meerdere instanties van de backend services.
- **CDN (Content Delivery Network):** Statische assets (afbeeldingen, CSS, JavaScript) zullen via een CDN worden geserveerd om de laadtijd voor gebruikers wereldwijd te verkorten.
- **API Gateway:** Een API Gateway kan worden gebruikt om requests te routeren, authenticatie te centraliseren en rate limiting te implementeren, wat bijdraagt aan de schaalbaarheid en beveiliging.

11. Test Strategy

Unit Tests

- **HomePage render:** Verifieert dat de homepage correct wordt gerenderd met alle verwachte componenten.
- **ProductList render:** Test de correcte weergave van de productlijst, inclusief de weergave van individuele producten.
- **ProductFilter render:** Valideert de functionaliteit en weergave van de productfilters (bv. categorie, prijs, maat).
- **Pagination render:** Controleert de correcte weergave en functionaliteit van de paginatiecomponenten voor productlijsten.
- **ProductDetailPage render:** Verifieert de correcte weergave van de productdetailpagina, inclusief productinformatie, afbeeldingen en opties.
- **AddToCartForm render:** Test de weergave en initiële staat van het formulier voor het toevoegen van producten aan het winkelmandje.
- **CartPage render:** Valideert de correcte weergave van de winkelmandpagina, inclusief de lijst met items, sub totaal en totaalprijs.
- **CartItem render:** Test de weergave van een individueel item in het winkelmandje, inclusief productnaam, prijs, hoeveelheid en verwijderoptie.
- **CheckoutButton render:** Controleert de weergave en functionaliteit van de knop om naar de checkout te gaan.
- **CheckoutPage render:** Verifieert de correcte weergave van de checkoutpagina, inclusief secties voor verzending, betaling en besteloverzicht.
- **ShippingForm render:** Test de weergave en validatie van het verzendinformatieformulier.
- **OrderSummary render:** Valideert de correcte weergave van het besteloverzicht tijdens de checkout.
- **PaymentForm render:** Test de weergave en validatie van het betaalinformatieformulier.
- **PlaceOrderButton render:** Controleert de weergave en functionaliteit van de knop om de bestelling te plaatsen.
- **OrderHistoryPage render:** Verifieert de correcte weergave van de bestelgeschiedenispagina.
- **OrderList render:** Test de weergave van de lijst met bestellingen in de bestelgeschiedenis.

- **OrderItem render:** Valideert de weergave van een individuele bestelling in de bestelgeschiedenis.
- **OrderDetailPage render:** Controleert de correcte weergave van de detailpagina van een specifieke bestelling.
- **ReturnRequestForm render:** Test de weergave en validatie van het formulier voor het aanvragen van een retour.
- **AdminProductListPage render:** Verifieert de correcte weergave van de lijst met producten in het adminpaneel.
- **AdminProductTable render:** Test de weergave van de tabel met producten in het adminpaneel, inclusief bewerkings- en verwijderopties.
- **AddProductButton render:** Controleert de weergave en functionaliteit van de knop om een nieuw product toe te voegen in het adminpaneel.
- **AdminAddProductPage render:** Verifieert de correcte weergave van de pagina voor het toevoegen van een nieuw product in het adminpaneel.
- **AdminProductForm render:** Test de weergave en validatie van het formulier voor het toevoegen/bewerken van producten in het adminpaneel.
- **AdminEditProductPage render:** Verifieert de correcte weergave van de pagina voor het bewerken van een product in het adminpaneel.
- **AdminOrderListPage render:** Verifieert de correcte weergave van de lijst met bestellingen in het adminpaneel.
- **AdminOrderTable render:** Test de weergave van de tabel met bestellingen in het adminpaneel, inclusief statusupdates.
- **AdminOrderDetailPage render:** Controleert de correcte weergave van de detailpagina van een specifieke bestelling in het adminpaneel.
- **UpdateOrderStatusForm render:** Test de weergave en functionaliteit van het formulier voor het updaten van de bestelstatus in het adminpaneel.
- **LoginPage render:** Verifieert de correcte weergave van de inlogpagina.
- **LoginForm render:** Test de weergave en validatie van het inlogformulier.
- **RegisterPage render:** Verifieert de correcte weergave van de registratiepagina.
- **RegisterForm render:** Test de weergave en validatie van het registratieformulier.
- **ErrorDisplay render:** Controleert de correcte weergave van foutmeldingen aan de gebruiker.
- **LoadingSpinner render:** Test de weergave van de laadindicator tijdens asynchrone operaties.

Integration Tests

- **GET /api/clothes → 200 OK:** Verifieert dat een GET-verzoek naar de API-endpoint voor kleding succesvol is en een 200 OK-status retourneert.
- **GET /api/clothes/{id} → 200 OK:** Test de succesvolle retrieval van een specifiek kledingitem via zijn ID.
- **POST /api/auth/register → 201 Created:** Valideert dat een succesvolle registratie van een nieuwe gebruiker resulteert in een 201 Created-status.

- **POST /api/auth/login → 201 Created:** Controleert of een succesvolle inlogpoging resulteert in een 201 Created-status.
- **GET /api/cart → 200 OK:** Test de succesvolle retrieval van de inhoud van het winkelmandje.
- **POST /api/cart/items → 201 Created:** Verifieert dat het toevoegen van een item aan het winkelmandje succesvol is en een 201 Created-status retourneert.
- **PATCH /api/cart/items/{id} → 200 OK:** Test de succesvolle update van een item in het winkelmandje (bv. hoeveelheid).
- **DELETE /api/cart/items/{id} → 204 No Content:** Controleert of het verwijderen van een item uit het winkelmandje succesvol is en een 204 No Content-status retourneert.
- **POST /api/orders → 201 Created:** Valideert dat het plaatsen van een nieuwe bestelling succesvol is en een 201 Created-status retourneert.
- **GET /api/orders/me → 200 OK:** Test de succesvolle retrieval van de bestelgeschiedenis van de ingelogde gebruiker.
- **POST /api/returns → 201 Created:** Verifieert dat het indienen van een retouraanvraag succesvol is en een 201 Created-status retourneert.
- **POST /api/admin/clothes → 201 Created:** Test de succesvolle toevoeging van een nieuw kledingitem via het admin API-endpoint.
- **PUT /api/admin/clothes/{id} → 200 OK:** Controleert de succesvolle update van een bestaand kledingitem via het admin API-endpoint.
- **PATCH /api/admin/orders/{id}/status → 200 OK:** Valideert de succesvolle update van de status van een bestelling via het admin API-endpoint.

E2E Tests

- **Gebruiker navigeert naar de homepage, filtert producten op prijs en voegt een product toe aan het winkelmandje:** Simuleert een gebruiker die de website verkent, producten filtert op basis van prijsbereik en vervolgens een geselecteerd product aan zijn winkelmandje toevoegt.
- **Gebruiker navigeert naar het winkelmandje, gaat door naar checkout, vult verzend- en betaalgegevens in en plaatst een bestelling:** Test het volledige aankoopproces, van het bekijken van het winkelmandje tot het succesvol afronden van een bestelling met geldige verzend- en betaalgegevens.
- **Gebruiker logt in, bekijkt de bestelgeschiedenis en dient een retouraanvraag in voor een specifieke bestelling:** Valideert de functionaliteit van gebruikersauthenticatie, het inzien van eerdere bestellingen en het initiëren van een retourproces voor een specifieke order.
- **Admin logt in, voegt een nieuw kledingitem toe, bewerkt een bestaand item en update de status van een bestelling:** Test de administratieve functies, waaronder het beheren van producten (toevoegen en bewerken) en het wijzigen van de status van geplaatste bestellingen.

12. Acceptance Criteria

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-001-1	REQ-001	Er zijn actieve kledingproducten beschikbaar in de database.	De gebruiker navigeert naar de productoverzichtspagina.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lijst van kledingproducten, waarbij elk product minimaal een 'id', 'name', 'price' en 'category' bevat.	integration
AC-001-2	REQ-001	Er zijn geen actieve kledingproducten beschikbaar in de database.	De gebruiker navigeert naar de productoverzichtspagina.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lege lijst van kledingproducten.	integration
AC-002-1	REQ-002	Er zijn kledingproducten met verschillende categorieën, maten, kleuren en prijzen beschikbaar.	De gebruiker past een filter toe op categorie 'Tops' en maat 'M'.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lijst van kledingproducten die voldoen aan de filtercriteria (categorie 'Tops' en maat 'M').	integration
AC-002-2	REQ-002	Er zijn kledingproducten met verschillende categorieën, maten, kleuren en prijzen beschikbaar.	De gebruiker past een filter toe op prijsbereik van €20 tot €50.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lijst van kledingproducten waarvan de prijs tussen €20 en €50 (inclusief) ligt.	integration
AC-002-3	REQ-002	Er zijn kledingproducten met verschillende categorieën, maten, kleuren en prijzen beschikbaar.	De gebruiker past een filter toe op een niet-bestaande categorie 'Schoenen'.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lege lijst van kledingproducten.	integration
AC-002-4	REQ-002	Er zijn kledingproducten met verschillende categorieën, maten, kleuren	De gebruiker past een filter toe op een maat die niet beschikbaar is voor enig product, bijvoorbeeld 'XXL'.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lege lijst van kledingproducten.	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
		en prijzen beschikbaar.			
AC-003-1	REQ-003	Er is een specifiek kledingproduct met ID 'prod-123' beschikbaar in de database met voorraad.	De gebruiker klikt op het kledingproduct met ID 'prod-123' in het overzicht.	De API GET /api/clothes/prod-123 retourneert een HTTP 200 statuscode met een 'ClothingItemDetailResponse' die de 'id', 'name', 'description', 'price', 'availableSizes', 'availableColors' en 'inventory' van het product bevat.	integration
AC-003-2	REQ-003	Er is geen kledingproduct met ID 'prod-999' in de database.	De gebruiker probeert de detailpagina te openen voor een niet-bestaand product met ID 'prod-999'.	De API GET /api/clothes/prod-999 retourneert een HTTP 404 statuscode met een 'ApiError'.	integration
AC-003-3	REQ-003	Er is een specifiek kledingproduct met ID 'prod-456' beschikbaar in de database, maar de voorraad voor alle maten is 0.	De gebruiker bekijkt de detailpagina van product 'prod-456'.	De 'inventory' velden voor alle maten van product 'prod-456' tonen een voorraadstatus van 0.	integration
AC-004-1	REQ-004	De gebruiker is ingelogd en het product 'prod-789' is beschikbaar met maat 'L' en kleur 'Blauw' en heeft een voorraad van minimaal 1.	De gebruiker selecteert maat 'L', kleur 'Blauw' voor product 'prod-789' en klikt op 'Toevoegen aan winkelmandje'.	De API POST /api/cart/items met body {'clothingItemId': 'prod-789', 'size': 'L', 'color': 'Blauw', 'quantity': 1} retourneert een HTTP 201 statuscode met een 'CartItemResponse' die het toegevoegde item bevat.	integration
AC-004-2	REQ-004	De gebruiker is ingelogd en het product 'prod-789' is beschikbaar met maat 'L' en kleur 'Blauw', maar de voorraad is 0.	De gebruiker selecteert maat 'L', kleur 'Blauw' voor product 'prod-789' en probeert op 'Toevoegen aan winkelmandje' te klikken.	De knop 'Toevoegen aan winkelmandje' is disabled.	e2e

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-004-3	REQ-004	De gebruiker is ingelogd en het product 'prod-789' is beschikbaar met maat 'L' en kleur 'Blauw' en heeft een voorraad van 1.	De gebruiker selecteert maat 'L', kleur 'Blauw' voor product 'prod-789' en klikt op 'Toevoegen aan winkelmandje' twee keer.	De API POST /api/cart/items met body {'clothingItemId': 'prod-789', 'size': 'L', 'color': 'Blauw', 'quantity': 1} wordt twee keer aangeroepen, en de bestaande cart item wordt geüpdatet naar quantity 2.	integration
AC-004-4	REQ-004	De gebruiker is ingelogd en het product 'prod-789' is beschikbaar met maat 'L' en kleur 'Blauw' en heeft een voorraad van 1.	De gebruiker probeert maat 'XL' toe te voegen aan het winkelmandje voor product 'prod-789', terwijl maat 'XL' niet beschikbaar is.	De API POST /api/cart/items met body {'clothingItemId': 'prod-789', 'size': 'XL', 'color': 'Blauw', 'quantity': 1} retourneert een HTTP 404 of 409 statuscode met een 'ApiError'.	integration
AC-005-1	REQ-005	De gebruiker is ingelogd en heeft minimaal één item in het winkelmandje.	De gebruiker doorloopt het checkout proces, vult geldige verzendgegevens in, selecteert een betaalmethode en bevestigt de bestelling.	De API POST /api/orders met geldige verzendgegevens en betaalmethode retourneert een HTTP 201 statuscode met een 'OrderResponse' die de 'id', 'status' (initieel 'PENDING'), 'totalPrice' en 'shippingAddress' van de bestelling bevat.	integration
AC-005-2	REQ-005	De gebruiker is ingelogd en heeft minimaal één item in het winkelmandje.	De gebruiker probeert het checkout proces te doorlopen zonder verzendgegevens in te vullen.	De API POST /api/orders retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat verzendgegevens verplicht zijn.	integration
AC-005-3	REQ-005	De gebruiker is ingelogd en heeft minimaal één item in het winkelmandje.	De gebruiker probeert het checkout proces te doorlopen met een ongeldige betaalmethode.	De API POST /api/orders retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat de betaalmethode ongeldig is.	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-005-4	REQ-005	De gebruiker is ingelogd en het winkelmandje is leeg.	De gebruiker probeert het checkout proces te starten.	De checkout functionaliteit is niet toegankelijk of de API POST /api/orders retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat het winkelmandje leeg is.	integration
AC-006-1	REQ-006	De gebruiker is ingelogd en heeft eerder bestellingen geplaatst.	De gebruiker navigeert naar de bestelgeschiedenispagina.	De API GET /api/orders/me retourneert een HTTP 200 statuscode met een 'OrderListResponse' die een lijst van de bestellingen van de gebruiker bevat, inclusief 'id', 'createdAt', 'status' en 'totalPrice'.	integration
AC-006-2	REQ-006	De gebruiker is ingelogd en heeft nog geen bestellingen geplaatst.	De gebruiker navigeert naar de bestelgeschiedenispagina.	De API GET /api/orders/me retourneert een HTTP 200 statuscode met een lege 'OrderListResponse'.	integration
AC-006-3	REQ-006	De gebruiker is ingelogd en heeft een bestelling met ID 'order-abc' met status 'PENDING'.	De gebruiker bekijkt de details van bestelling 'order-abc' in de bestelgeschiedenis.	De status van bestelling 'order-abc' wordt correct weergegeven als 'PENDING'.	integration
AC-007-1	REQ-007	De gebruiker is ingelogd met de rol 'Admin'.	De admin voegt een nieuw kledingproduct toe via de admin interface.	De API POST /api/admin/clothes met geldige productgegevens retourneert een HTTP 201 statuscode met een 'ClothingItemResponse' van het aangemaakte product.	integration
AC-007-2	REQ-007	De gebruiker is ingelogd met de rol 'Admin'.	De admin wijzigt de prijs van een bestaand kledingproduct met ID 'prod-xyz'.	De API PUT /api/admin/clothes/prod-xyz met een nieuwe prijs retourneert een HTTP 200 statuscode met een 'ClothingItemResponse' van het bijgewerkte product.	integration
AC-007-3	REQ-007	De gebruiker is ingelogd met de rol 'Admin'.	De admin probeert een product toe te voegen met een ongeldige prijs (bv. negatief getal).	De API POST /api/admin/clothes retourneert een HTTP 400	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
				statuscode met een 'ApiError'.	
AC-007-4	REQ-007	Er zijn bestaande orders in het systeem.	De admin bekijkt de lijst met orders.	Er is een endpoint beschikbaar voor de admin om alle orders te bekijken (impliciet via de admin rol check op andere endpoints, of een expliciet GET /api/admin/orders endpoint indien beschikbaar).	integration
AC-008-1	REQ-008	De gebruiker is ingelogd met de rol 'Admin' en er is een bestelling met ID 'order-def' die geleverd is.	De admin opent de details van bestelling 'order-def'.	De admin kan de details van bestelling 'order-def' bekijken, inclusief de orderregels, verzendgegevens en status.	integration
AC-008-2	REQ-008	De gebruiker is ingelogd met de rol 'Admin' en er is een bestelling met ID 'order-ghi' die nog niet geleverd is.	De admin probeert een retouraanvraag aan te maken voor bestelling 'order-ghi'.	De API POST /api/returns met orderId 'order-ghi' retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat een retour alleen mogelijk is voor geleverde bestellingen.	integration
AC-008-3	REQ-008	De gebruiker is ingelogd met de rol 'Admin' en er is een bestelling met ID 'order-jkl' die geleverd is.	De admin maakt een retouraanvraag aan voor bestelling 'order-jkl' met reden 'Product beschadigd'.	De API POST /api/returns met body {'orderId': 'order-jkl', 'reason': 'Product beschadigd'} retourneert een HTTP 201 statuscode met een 'ReturnRequestResponse' die de aangemaakte retouraanvraag bevat met status 'PENDING'.	integration
AC-009-1	REQ-009	De gebruiker is ingelogd en heeft een geleverde bestelling met ID 'order-mno' die binnen 14 dagen na levering is.	De gebruiker dient een retouraanvraag in voor bestelling 'order-mno' met reden 'Niet tevreden'.	De API POST /api/returns met body {'orderId': 'order-mno', 'reason': 'Niet tevreden'} retourneert een HTTP 201 statuscode met een 'ReturnRequestResponse' die de retouraanvraag bevat met status 'PENDING'.	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-009-2	REQ-009	De gebruiker is ingelogd en heeft een geleverde bestelling met ID 'order-pqr' die meer dan 14 dagen geleden geleverd is.	De gebruiker probeert een retouraanvraag in te dienen voor bestelling 'order-pqr'.	De API POST /api/returns met orderId 'order-pqr' retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat de retourtermijn is verstreken.	integration
AC-009-3	REQ-009	De gebruiker is ingelogd en heeft een bestelling met ID 'order-stu' die nog niet geleverd is.	De gebruiker probeert een retouraanvraag in te dienen voor bestelling 'order-stu'.	De API POST /api/returns met orderId 'order-stu' retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat een retour alleen mogelijk is voor geleverde bestellingen.	integration
AC-010-1	REQ-010	Product 'prod-abc' is beschikbaar met maat 'M' en kleur 'Rood', en de voorraad voor maat 'M' is 5.	De gebruiker probeert product 'prod-abc' met maat 'M' en kleur 'Rood' toe te voegen aan het winkelmandje met quantity 1.	De API POST /api/cart/items met body {'clothingItemId': 'prod-abc', 'size': 'M', 'color': 'Rood', 'quantity': 1} retourneert een HTTP 201 statuscode.	integration
AC-010-2	REQ-010	Product 'prod-abc' is beschikbaar met maat 'M' en kleur 'Rood', maar de voorraad voor maat 'M' is 0.	De gebruiker probeert product 'prod-abc' met maat 'M' en kleur 'Rood' toe te voegen aan het winkelmandje met quantity 1.	De API POST /api/cart/items met body {'clothingItemId': 'prod-abc', 'size': 'M', 'color': 'Rood', 'quantity': 1} retourneert een HTTP 409 statuscode met een 'ApiError' die aangeeft dat het product niet op voorraad is.	integration
AC-010-3	REQ-010	Product 'prod-abc' is beschikbaar met maat 'M' en kleur 'Rood', en de voorraad voor maat 'M' is 5.	De gebruiker probeert product 'prod-abc' met maat 'M' en kleur 'Rood' toe te voegen aan het winkelmandje met quantity 6.	De API POST /api/cart/items met body {'clothingItemId': 'prod-abc', 'size': 'M', 'color': 'Rood', 'quantity': 6} retourneert een HTTP 409 statuscode met een 'ApiError' die aangeeft dat de gevraagde hoeveelheid de voorraad overschrijdt.	integration
AC-011-1	REQ-011	Product 'prod-xyz' is beschikbaar met maat 'L' en kleur 'Groen', en de	De gebruiker selecteert maat 'L' en kleur 'Groen' voor product 'prod-xyz'.	De knop 'Toevoegen aan winkelmandje' is enabled.	e2e

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
		voorraad voor deze combinatie is 3.			
AC-011-2	REQ-011	Product 'prod-xyz' is beschikbaar met maat 'L' en kleur 'Groen', maar de voorraad voor deze combinatie is 0.	De gebruiker selecteert maat 'L' en kleur 'Groen' voor product 'prod-xyz'.	De knop 'Toevoegen aan winkelmandje' is disabled.	e2e
AC-011-3	REQ-011	Product 'prod-xyz' is beschikbaar met maat 'L' en kleur 'Groen', en de voorraad voor deze combinatie is 3.	De gebruiker selecteert maat 'L' en kleur 'Blauw' voor product 'prod-xyz', waarbij deze combinatie niet op voorraad is.	De knop 'Toevoegen aan winkelmandje' is disabled.	e2e
AC-012-1	REQ-012	Een gebruiker plaatst een nieuwe bestelling.	De bestelling wordt succesvol aangemaakt via POST /api/orders.	De status van de bestelling is 'PENDING'.	integration
AC-012-2	REQ-012	Een bestelling met ID 'order-xyz' heeft de status 'PENDING'.	De betalingsprovider stuurt een succesvolle webhook callback voor bestelling 'order-xyz'.	De status van bestelling 'order-xyz' wordt bijgewerkt naar 'PAID'.	integration
AC-012-3	REQ-012	Een bestelling met ID 'order-abc' heeft de status 'PENDING'.	De betalingsprovider stuurt een mislukte webhook callback voor bestelling 'order-abc'.	De status van bestelling 'order-abc' blijft 'PENDING' (of wordt bijgewerkt naar een relevante 'FAILED' status indien gedefinieerd).	integration
AC-013-1	REQ-013	Product 'prod-111' heeft voorraad van 5 stuks voor maat 'S' en kleur 'Zwart'.	Een gebruiker plaatst een bestelling voor 2 stuks van product 'prod-111' met maat 'S' en kleur 'Zwart'.	De voorraad voor product 'prod-111', maat 'S', kleur 'Zwart' wordt gereserveerd (bv. de beschikbare voorraad wordt 3, en een aparte 'gereserveerde' teller wordt 2).	integration
AC-013-2	REQ-013	Een bestelling met gereserveerde voorraad voor	De betalingswebhook voor de bestelling wordt ontvangen.	De definitieve voorraad voor product 'prod-111', maat 'S', kleur 'Zwart' wordt	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
		product 'prod-111', maat 'S', kleur 'Zwart' (2 stuks) wordt succesvol betaald.		verminderd met 2 (van 3 naar 1).	
AC-013-3	REQ-013	Een bestelling met gereserveerde voorraad voor product 'prod-222', maat 'M', kleur 'Blauw' (1 stuk) wordt geannuleerd of de betaling mislukt.	De betalingswebhook voor de bestelling wordt niet ontvangen of is een mislukte callback.	De gereserveerde voorraad voor product 'prod-222', maat 'M', kleur 'Blauw' wordt vrijgegeven (de voorraad keert terug naar de oorspronkelijke waarde vóór reservering).	integration
AC-014-1	REQ-014	De gebruiker is ingelogd en heeft een bestelling met ID 'order-xyz'.	De gebruiker navigeert naar de bestelgeschiedenis.	De gebruiker kan de details van bestelling 'order-xyz' bekijken.	integration
AC-014-2	REQ-014	De gebruiker is ingelogd.	De gebruiker probeert toegang te krijgen tot een admin-specifiek order endpoint (bv. het wijzigen van een orderstatus).	De API retourneert een HTTP 403 statuscode (Forbidden) of 401 (Unauthorized) indien niet ingelogd.	integration
AC-015-1	REQ-015	De gebruiker is ingelogd met de rol 'Admin'.	De gebruiker probeert toegang te krijgen tot het endpoint POST /api/admin/clothes.	De API retourneert een HTTP 201 statuscode met de verwachte response.	integration
AC-015-2	REQ-015	De gebruiker is ingelogd met de rol 'USER'.	De gebruiker probeert toegang te krijgen tot het endpoint POST /api/admin/clothes.	De API retourneert een HTTP 403 statuscode (Forbidden).	integration
AC-015-3	REQ-015	De gebruiker is niet ingelogd.	De gebruiker probeert toegang te krijgen tot het endpoint POST /api/admin/clothes.	De API retourneert een HTTP 401 statuscode (Unauthorized).	integration
AC-016-1	REQ-016	De gebruiker is ingelogd en heeft een bestelling met ID 'order-del-1' die 5	De gebruiker dient een retouraanvraag in voor bestelling 'order-del-1'.	De API POST /api/returns met orderId 'order-del-1' retourneert een HTTP 201 statuscode.	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
		dagen geleden geleverd is.			
AC-016-2	REQ-016	De gebruiker is ingelogd en heeft een bestelling met ID 'order-del-2' die 15 dagen geleden geleverd is.	De gebruiker probeert een retouraanvraag in te dienen voor bestelling 'order-del-2'.	De API POST /api/returns met orderId 'order-del-2' retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat de retourtermijn is verstreken.	integration
AC-016-3	REQ-016	De gebruiker is ingelogd en heeft een bestelling met ID 'order-not-del' die nog niet geleverd is.	De gebruiker probeert een retouraanvraag in te dienen voor bestelling 'order-not-del'.	De API POST /api/returns met orderId 'order-not-del' retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat een retour alleen mogelijk is voor geleverde bestellingen.	integration
AC-017-1	REQ-017	Er zijn actieve kledingproducten beschikbaar in de database.	De gebruiker navigeert naar de productoverzichtspagina.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lijst van kledingproducten, waarbij elk product minimaal een 'id', 'name', 'price' en 'category' bevat.	integration
AC-017-2	REQ-017	Er zijn geen actieve kledingproducten beschikbaar in de database.	De gebruiker navigeert naar de productoverzichtspagina.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lege lijst van kledingproducten.	integration
AC-018-1	REQ-018	Er zijn kledingproducten met verschillende categorieën, maten, kleuren en prijzen beschikbaar.	De gebruiker past een filter toe op categorie 'Tops' en maat 'M'.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lijst van kledingproducten die voldoen aan de filtercriteria (categorie 'Tops' en maat 'M').	integration
AC-018-2	REQ-018	Er zijn kledingproducten met verschillende categorieën, maten, kleuren en prijzen beschikbaar.	De gebruiker past een filter toe op prijsbereik van €20 tot €50.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lijst van kledingproducten waarvan de prijs tussen €20 en €50 (inclusief) ligt.	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-018-3	REQ-018	Er zijn kledingproducten met verschillende categorieën, maten, kleuren en prijzen beschikbaar.	De gebruiker past een filter toe op een niet-bestaande categorie 'Schoenen'.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lege lijst van kledingproducten.	integration
AC-018-4	REQ-018	Er zijn kledingproducten met verschillende categorieën, maten, kleuren en prijzen beschikbaar.	De gebruiker past een filter toe op een maat die niet beschikbaar is voor enig product, bijvoorbeeld 'XXL'.	De API GET /api/clothes retourneert een HTTP 200 statuscode met een lege lijst van kledingproducten.	integration
AC-019-1	REQ-019	Er is een specifiek kledingproduct met ID 'prod-123' beschikbaar in de database met voorraad.	De gebruiker klikt op het kledingproduct met ID 'prod-123' in het overzicht.	De API GET /api/clothes/prod-123 retourneert een HTTP 200 statuscode met een 'ClothingItemDetailResponse' die de 'id', 'name', 'description', 'price', 'availableSizes', 'availableColors' en 'inventory' van het product bevat.	integration
AC-019-2	REQ-019	Er is geen kledingproduct met ID 'prod-999' in de database.	De gebruiker probeert de detailpagina te openen voor een niet-bestaand product met ID 'prod-999'.	De API GET /api/clothes/prod-999 retourneert een HTTP 404 statuscode met een 'ApiError'.	integration
AC-019-3	REQ-019	Er is een specifiek kledingproduct met ID 'prod-456' beschikbaar in de database, maar de voorraad voor alle maten is 0.	De gebruiker bekijkt de detailpagina van product 'prod-456'.	De 'inventory' velden voor alle maten van product 'prod-456' tonen een voorraadstatus van 0.	integration
AC-020-1	REQ-020	De gebruiker is ingelogd en het product 'prod-789' is	De gebruiker selecteert maat 'L', kleur 'Blauw' voor product 'prod-789'	De API POST /api/cart/items met body {'clothingItemId': 'prod-789', 'size': 'L', 'color': 'Blauw', 'quantity': 1}	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
		beschikbaar met maat 'L' en kleur 'Blauw' en heeft een voorraad van minimaal 1.	en klikt op 'Toevoegen aan winkelmandje'.	retourneert een HTTP 201 statuscode met een 'CartItemResponse' die het toegevoegde item bevat.	
AC-020-2	REQ-020	De gebruiker is ingelogd en het product 'prod-789' is beschikbaar met maat 'L' en kleur 'Blauw', maar de voorraad is 0.	De gebruiker selecteert maat 'L', kleur 'Blauw' voor product 'prod-789' en probeert op 'Toevoegen aan winkelmandje' te klikken.	De knop 'Toevoegen aan winkelmandje' is disabled.	e2e
AC-020-3	REQ-020	De gebruiker is ingelogd en het product 'prod-789' is beschikbaar met maat 'L' en kleur 'Blauw' en heeft een voorraad van 1.	De gebruiker selecteert maat 'L', kleur 'Blauw' voor product 'prod-789' en klikt op 'Toevoegen aan winkelmandje' twee keer.	De API POST /api/cart/items met body {'clothingItemId': 'prod-789', 'size': 'L', 'color': 'Blauw', 'quantity': 1} wordt twee keer aangeroepen, en de bestaande cart item wordt geüpdatet naar quantity 2.	integration
AC-020-4	REQ-020	De gebruiker is ingelogd en het product 'prod-789' is beschikbaar met maat 'L' en kleur 'Blauw' en heeft een voorraad van 1.	De gebruiker probeert maat 'XL' toe te voegen aan het winkelmandje voor product 'prod-789', terwijl maat 'XL' niet beschikbaar is.	De API POST /api/cart/items met body {'clothingItemId': 'prod-789', 'size': 'XL', 'color': 'Blauw', 'quantity': 1} retourneert een HTTP 404 of 409 statuscode met een 'ApiError'.	integration
AC-021-1	REQ-021	De gebruiker is ingelogd en heeft minimaal één item in het winkelmandje.	De gebruiker doorloopt het checkout proces, vult geldige verzendgegevens in, selecteert een betaalmethode en bevestigt de bestelling.	De API POST /api/orders met geldige verzendgegevens en betaalmethode retourneert een HTTP 201 statuscode met een 'OrderResponse' die de 'id', 'status' (initieel 'PENDING'), 'totalPrice' en 'shippingAddress' van de bestelling bevat.	integration
AC-021-2	REQ-021	De gebruiker is ingelogd en heeft minimaal één item in het winkelmandje.	De gebruiker probeert het checkout proces te doorlopen zonder verzendgegevens in te vullen.	De API POST /api/orders retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat	integration

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
				verzendinggegevens verplicht zijn.	
AC-021-3	REQ-021	De gebruiker is ingelogd en heeft minimaal één item in het winkelmandje.	De gebruiker probeert het checkout proces te doorlopen met een ongeldige betaalmethode.	De API POST /api/orders retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat de betaalmethode ongeldig is.	integration
AC-021-4	REQ-021	De gebruiker is ingelogd en het winkelmandje is leeg.	De gebruiker probeert het checkout proces te starten.	De checkout functionaliteit is niet toegankelijk of de API POST /api/orders retourneert een HTTP 400 statuscode met een 'ApiError' die aangeeft dat het winkelmandje leeg is.	integration
AC-022-1	REQ-022	De gebruiker is ingelogd en heeft eerder bestellingen geplaatst.	De gebruiker navigeert naar de bestelgeschiedenispagina.	De API GET /api/orders/me retourneert een HTTP 200 statuscode met een 'OrderListResponse' die een lijst van de bestellingen van de gebruiker bevat, inclusief 'id', 'createdAt', 'status' en 'totalPrice'.	integration
AC-022-2	REQ-022	De gebruiker is ingelogd en heeft nog geen bestellingen geplaatst.	De gebruiker navigeert naar de bestelgeschiedenispagina.	De API GET /api/orders/me retourneert een HTTP 200 statuscode met een lege 'OrderListResponse'.	integration
AC-022-3	REQ-022	De gebruiker is ingelogd en heeft een bestelling met ID 'order-abc' met status 'PENDING'.	De gebruiker bekijkt de details van bestelling 'order-abc' in de bestelgeschiedenis.	De status van bestelling 'order-abc' wordt correct weergegeven als 'PENDING'.	integration
AC-023-1	REQ-023	De applicatie wordt geopend op een desktop browser.	De gebruiker navigeert door de productcatalogus en bekijkt productdetails.	De layout van de pagina past zich correct aan de desktop resolutie aan, met alle elementen goed zichtbaar en bruikbaar.	e2e
AC-023-2	REQ-023	De applicatie wordt geopend op een tablet browser.	De gebruiker navigeert door de productcatalogus en bekijkt productdetails.	De layout van de pagina past zich correct aan de tablet resolutie aan, met alle elementen goed zichtbaar en bruikbaar.	e2e

AC-ID	REQ	Gegeven	Wanneer	Dan	Testtype
AC-023-3	REQ-023	De applicatie wordt geopend op een mobiele browser.	De gebruiker navigeert door de productcatalogus en bekijkt productdetails.	De layout van de pagina past zich correct aan de mobiele resolutie aan, met alle elementen goed zichtbaar en bruikbaar.	e2e
AC-024-1	REQ-024	Een gebruiker registreert zich met een nieuw wachtwoord.	Het registratieproces wordt voltooid.	Het wachtwoord van de gebruiker wordt gehasht opgeslagen in de database, niet in platte tekst.	integration
AC-024-2	REQ-024	Een gebruiker logt in met een bestaand wachtwoord.	Het login proces wordt uitgevoerd.	Het ingevoerde wachtwoord wordt gehasht en vergeleken met de gehashte waarde in de database, zonder het originele wachtwoord te onthullen.	integration

13. Traceability Matrix

REQ	Backend	Frontend	Tests
REQ-001	ClothingItemController, ClothingItemService, ClothingItemRepository	HomePage, ProductList	Verifieer dat de productlijst op de homepage en productpagina actieve kledingproducten toont.
REQ-002	ClothingItemController, ClothingItemService	ProductFilter	Test filtering op categorie, maat, kleur, prijs en beschikbaarheid op de productpagina.
REQ-003	ClothingItemController, ClothingItemService, ClothingItemRepository	ProductDetailPage	Controleer of de productdetailpagina correcte prijs, beschrijving, beschikbare maten en voorraadstatus toont.
REQ-004	CartController, CartService, CartItemRepository, ClothingItemService, InventoryService	ProductDetailPage, AddToCartForm	Valideer dat een product met gekozen maat en kleur succesvol aan het winkelmandje kan worden toegevoegd.

REQ	Backend	Frontend	Tests
REQ-005	OrderController, OrderService, PaymentController, PaymentService, AddressService	CheckoutPage, ShippingForm, OrderSummary, PaymentForm, PlaceOrderButton	Test het volledige checkoutproces inclusief verzendgegevens, orderoverzicht en betalingsmethode.
REQ-006	OrderController, OrderService, OrderRepository	OrderHistoryPage, OrderList, OrderItem	Controleer of de klant zijn bestelgeschiedenis en de status van zijn bestellingen kan bekijken.
REQ-007	ClothingItemController, CategoryController, OrderController, ClothingItemService, CategoryService, OrderService	AdminProductListPage, AdminProductTable, AdminAddProductPage, AdminEditProductPage, AdminOrderListPage, AdminOrderTable	Verifieer dat de admin producten, categorieën en orders kan beheren via de admin interface.
REQ-008	OrderController, OrderService, ReturnController, ReturnService	AdminOrderDetailPage, UpdateOrderStatusForm	Test of de admin bestellingen kan openen en retouraanvragen kan aanmaken.
REQ-009	ReturnController, ReturnService, OrderService	OrderHistoryPage, OrderItem, ReturnRequestForm	Valideer dat een klant een retouraanvraag kan indienen voor een geleverde bestelling.
REQ-010	OrderService, InventoryService, ClothingItemValidator		Test dat een product niet besteld kan worden als de gekozen maat niet op voorraad is.
REQ-011	ClothingItemController, InventoryService	ProductDetailPage, AddToCartForm	Controleer of de 'Toevoegen aan winkelmand' knop disabled is wanneer de gekozen maat/kleur niet beschikbaar is.
REQ-012	OrderService, PaymentService		Verifieer dat een bestelling de status 'Pending' krijgt en pas 'Paid' wordt na een succesvolle webhook van de betalingsprovider.
REQ-013	OrderService, InventoryService, PaymentService		Test dat voorraad wordt gereserveerd bij ordercreatie en definitief verminderd na succesvolle betaling.
REQ-014	OrderController, OrderService	OrderHistoryPage	Controleer of klanten alleen hun eigen bestellingen

REQ	Backend	Frontend	Tests
			kunnen bekijken.
REQ-015	SecurityConfig, JwtTokenProvider, AuthController	LoginPage, RegisterPage	Test dat admin endpoints alleen toegankelijk zijn voor gebruikers met de rol Admin.
REQ-016	ReturnService, OrderService	OrderHistoryPage, OrderItem, ReturnRequestForm	Valideer dat retouraanvragen alleen mogelijk zijn voor bestellingen met status 'Delivered' en binnen 14 dagen na levering.
REQ-017	ClothingItemController, ClothingItemService, ClothingItemRepository	HomePage, ProductList	Verifieer dat de productlijst op de homepage en productpagina actieve kledingproducten toont.
REQ-018	ClothingItemController, ClothingItemService	ProductFilter	Test filtering op categorie, maat, kleur, prijs en beschikbaarheid op de productpagina.
REQ-019	ClothingItemController, ClothingItemService, ClothingItemRepository	ProductDetailPage	Controleer of de productdetailpagina correcte prijs, beschrijving, beschikbare maten en voorraadstatus toont.
REQ-020	CartController, CartService, CartItemRepository, ClothingItemService, InventoryService	ProductDetailPage, AddToCartForm	Valideer dat een product met gekozen maat en kleur succesvol aan het winkelmandje kan worden toegevoegd.
REQ-021	OrderController, OrderService, PaymentController, PaymentService, AddressService	CheckoutPage, ShippingForm, OrderSummary, PaymentForm, PlaceOrderButton	Test het volledige checkoutproces inclusief verzendgegevens, orderoverzicht en betalingsmethode.
REQ-022	OrderController, OrderService, OrderRepository	OrderHistoryPage, OrderList, OrderItem	Controleer of de klant zijn bestelgeschiedenis en de status van zijn bestellingen kan bekijken.
REQ-023		HomePage, ProductList, ProductDetailPage, CartPage, CheckoutPage, OrderHistoryPage, LoginPage, RegisterPage	Test de responsiviteit van de applicatie op desktop, tablet en mobiele apparaten.
REQ-024	PasswordHasher, AuthService, UserRepository	RegisterPage, LoginPage	Verifieer dat wachtwoorden gehasht worden opgeslagen

REQ	Backend	Frontend	Tests
			in de database.